Sprint 1 Requirements

1. **Create a master password on first run**

   Prompt the user to create an account password at least 16 characters long with specific character requirements: the inclusion of 1 non-alphanumeric character, 1 uppercase letter, and 1 lowercase letter. Have the user verify the password by typing it in a second time to confirm the same thing was entered twice. The system will only accept the password if the two entries match.

2. **Unlock the vault with the master password and reject wrong passwords without revealing details**

   Accept the correct password in a case-sensitive manner. Reject any entered passwords that are incorrect. When the correct password is entered, go into the user profile with correct user information. Present the home screen to the user for their further actions.

3. **Derive a separate encryption key for each data file from the master password using a slow, salted method**

   When a file is created, the system should generate a random salt and combine it with the master password to derive the file's encryption key. The salt and key derivation parameters should be stored, but the master password and derived keys should not be stored using plaintext. When accessing a file, the same parameters and salt should be used to derive the key again as when the file was created.

4. **Encrypt and decrypt each data file with authenticated encryption**

   Encrypt each data file with an authenticated encryption method before it's saved, using the key derived for that file. Use a new random nonce every time a file is saved so the same data never encrypts the same way twice. When the user unlocks the vault, decrypt

each file with its own key and check its authentication tag before any of the data is used. If the tag doesn't match (due to the key being wrong or the file being changed), stop and do not return any partial data. Only encrypted data is ever written to disk.

5. **Build with a Makefile that produces a single executable and provide a desktop launcher**

   The makefile should create a single executable, connecting the separate modules through a single user interface. The makefile should compile each source file into object files which are then consolidated to create the final executable. The executable starts with the MenuController which has the modules of the feature, storage, and security layers as dependencies.

6. **Make every feature accessible from the executable**

   The executable should be able to access every layer and their respective modules. Interactions with the modules start with the MenuController, which calls the other modules. Every module and associated features should be able to be accessed from the initial MenuController module.

7. **Detect tampered and corrupted files and fail safely**

   The system should detect tampered or corrupted files and prevent them from being processed. If a file fails an integrity check or cannot be read properly, the system should safely stop processing the file, display an error message, and avoid unintended behavior by returning to previous functionality.